

Ordinary Differential Equations: Initial Value Problems

Numerical Analysis E3/I3, FMN050

Numerical Analysis

Centre for Mathematical Sciences

Lund University, Sweden

URL: `http://www.maths.lth.se/na/`

May 15, 2006

Ordinary differential equations

Two basic problem types:

- **Initial value problems (IVP)**

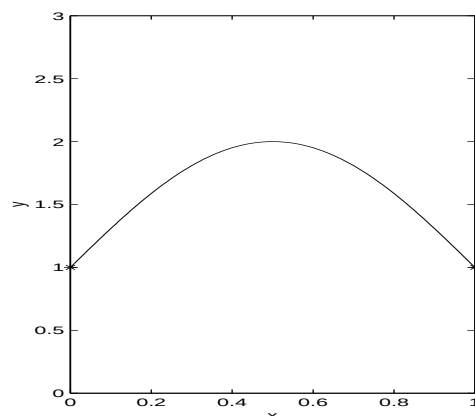
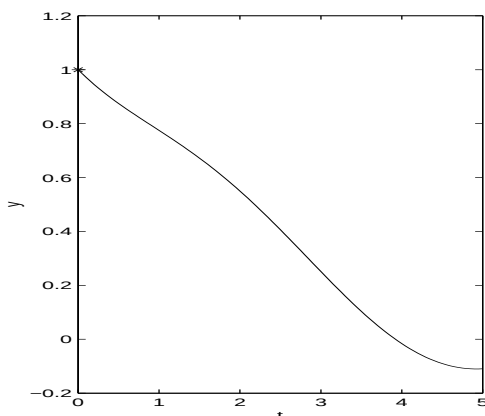
$$\dot{y} = f(t, y); \quad y(0) = y_0$$

Compute $y(t)$ for $t \geq 0$.

- **Boundary value problems (BVP)**

$$y'' = f(x, y, y'); \\ y(0) = y_0; \quad y(1) = y_1$$

Compute $y(x)$ for $x \in [0, 1]$.



Applications

Initial value problems:

- mechanics $M\ddot{q} = F(q)$
- electrical circuits $C\dot{v} = -I(v)$
- chemical reactions $\dot{c} = f(c)$
- . . .

Boundary value problems:

- materials $u'' = M(u)/EI$
- microphysics $-\frac{\hbar}{2m}\psi'' = E$
- eigenmodes $u'' + \lambda u = 0$
- . . .

Some nonlinear problems

Robertson, chemical kinetics

$$\dot{y}_1 = -(55 + y_3)y_1 + 65y_2$$

$$\dot{y}_2 = .0785(y_1 - y_2)$$

$$\dot{y}_3 = .1y_1$$

“Brusselator”, chemical kinetics

$$\dot{y}_1 = 1 + y_1^2 y_2 - 9.5y_1$$

$$\dot{y}_2 = 8.5y_1 - y_1^2 y_2$$

Liénard – van der Pol oscillator

$$\dot{y}_1 = y_2$$

$$\varepsilon \dot{y}_2 = -y_1 + y_2(1 - y_2^2)$$

More nonlinear problems

Lorenz system

$$\dot{y}_1 = -\sigma y_1 + \sigma y_2$$

$$\dot{y}_2 = Ry_1 - y_2 - y_1y_3$$

$$\dot{y}_3 = -by_3 + y_1y_2$$

Rössler system

$$\dot{y}_1 = -y_2 - y_3$$

$$\dot{y}_2 = y_1 + ay_2$$

$$\dot{y}_3 = b + y_1y_3 - cy_3$$

Belousov–Zhabotinskii reaction

$$\dot{y}_1 = -k_f y_1 y_2 + k_b y_3 - r(y_1 - y_1^0)$$

$$\dot{y}_2 = -k_f y_1 y_2 + k_b y_3 - r(y_2 - y_2^0)$$

$$\dot{y}_3 = k_f y_1 y_2 - k_b y_3 - r y_3$$

Initial value problems

Standard formulation

$$\dot{y} = f(t, y); \quad y(0) = y_0$$

Scalar equation: $y \in \mathbb{R}$; f scalar;

Systems of ODEs: $y \in \mathbb{R}^m$; f vector-valued.

Theorem: (Cauchy & Peano) A solution to $\dot{y} = f(t, y)$ exists if f is continuous.

Note: *Uniqueness is not guaranteed!* Example:

$$\dot{y} = 2\sqrt{y}; \quad y(0) = 0 \quad \text{with solution}$$

$$y(t) = 0, \quad t \leq \tau \quad \text{and} \quad y(t) = (t - \tau)^2, \quad t > \tau$$

This may happen in real physical models!

Consider the energy of a falling particle of mass m :

$$\frac{1}{2} m \dot{y}^2 + mgy = E := 0$$

This yields $\dot{y} = -\sqrt{2g} \sqrt{-y}$ with $y(0) = 0$.

Existence and uniqueness

Theorem: If $f(t, y)$ is continuous on $[0, T]$ and satisfies the *Lipschitz condition*

$$\|f(t, u) - f(t, v)\| \leq L[f] \cdot \|u - v\|$$

for all u, v and some constant $L[f] < \infty$ there exists a unique solution to the initial value problem

$$\dot{y} = f(t, y)$$

on $[0, T]$ for every initial value $y(0) = y_0$.

Note: Most problems do not satisfy a Lipschitz condition on *all* of $\mathbb{R}^m$.

$$\dot{y} = y^2; \quad y(0) = y_0 > 0$$

has solution

$$y(t) = \frac{y_0}{1 - y_0 t}$$

The solution blows up at $t = 1/y_0$.

Transformation to standard form

Example:

$$\ddot{y} = f(t, y, \dot{y}); \quad y(0) = y_0; \quad \dot{y}(0) = \dot{y}_0$$

Standard substitution:

Introduce

$$x_1 = y$$

$$x_2 = \dot{y}$$

Then

$$\dot{x}_1 = x_2$$

$$\dot{x}_2 = f(t, x_1, x_2),$$

with $x_1(0) = y_0$ and $x_2(0) = \dot{y}_0$.

Transformation to standard form. . .

Simple tricks often make it possible to solve “strange” problems by standard methods.

Example:

$$\frac{d}{dt} q(v) = -f(t, v); \quad v(0) = v_0$$

Introduce

$$w = q(v); \quad w_0 = q(v(0))$$

Then

$$\begin{aligned} \dot{w} &= -f(t, v) & w(0) &= q(v_0) \\ 0 &= w - q(v) & v(0) &= v_0 \end{aligned}$$

Standard differential-algebraic equation (DAE)

The Explicit Euler method (1768)

$$\dot{y} = f(t, y); \quad y(0) = y_0$$

Replace derivative by finite difference approximation

$$\dot{y}(t_n) \approx \frac{y(t_n + h) - y(t_n)}{h}$$

Let $\{y_n\}$ denote numerical approx. to $\{y(t_n)\}$ and let $h = t_{n+1} - t_n$ denote the time-step.

Then

$$\frac{y_{n+1} - y_n}{h} = f(t_n, y_n)$$

The Explicit Euler method:

$$y_{n+1} = y_n + hf(t_n, y_n)$$

$$t_{n+1} = t_n + h$$

Explicit Euler, alternative derivation

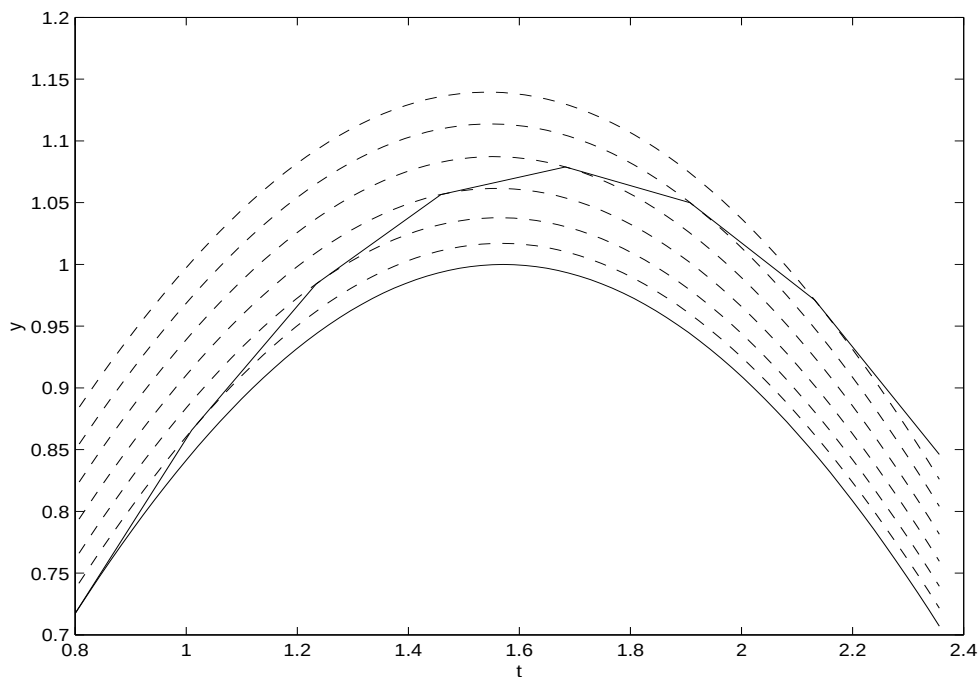
$$y(t_n + h) = y(t_n) + h\dot{y}(t_n) + O(h^2)$$

$$\approx y(t_n) + hf(t_n, y(t_n))$$

$\Rightarrow$

$$y_{n+1} = y_n + hf(t_n, y_n)$$

“Take a step of size h in the direction of the tangent.”



Each step introduces an error and ends up on a different solution trajectory (dashed curves).

Mean value theorem (scalar case)

$$\exists \xi \in [t_n, t_{n+1}] : \dot{y}(\xi) = \frac{y(t_{n+1}) - y(t_n)}{t_{n+1} - t_n}$$

“What is ξ ?”

The construction of time-stepping methods can be thought of as finding good approximations to ξ in the mean value theorem.

Case 1: $\xi = t_n$, *forward difference approximation*

$$\dot{y}(t_n) \approx \frac{y(t_n + h) - y(t_n)}{h}$$

yields the **Explicit Euler method**,

$$y_{n+1} = y_n + hf(t_n, y_n)$$

The implicit Euler method

Case 2: $\xi = t_{n+1}$, *backward difference approximation*

$$\dot{y}(t_n + h) \approx \frac{y(t_n + h) - y(t_n)}{h}$$

yields the **Implicit Euler method**,

$$y_{n+1} = y_n + hf(t_{n+1}, y_{n+1})$$

Note:

- **Equation solving:** y_{n+1} is *implicitly defined*.
- **Accuracy:** Explicit and implicit Euler have *comparable accuracy*.
- **Stability:** Implicit Euler has *better stability*.
- **Computational expense:** Trade equation solving costs for *much larger time-steps*.

Other simple methods

Case 3: $\xi = t_n + h/2$, *symmetric difference quotient*

$$\dot{y} \left(t_n + \frac{h}{2} \right) \approx \frac{y(t_n + h) - y(t_n)}{h}$$

Trapezoidal Rule

$$y_{n+1} = y_n + h \left(\frac{f(t_n, y_n)}{2} + \frac{f(t_{n+1}, y_{n+1})}{2} \right)$$

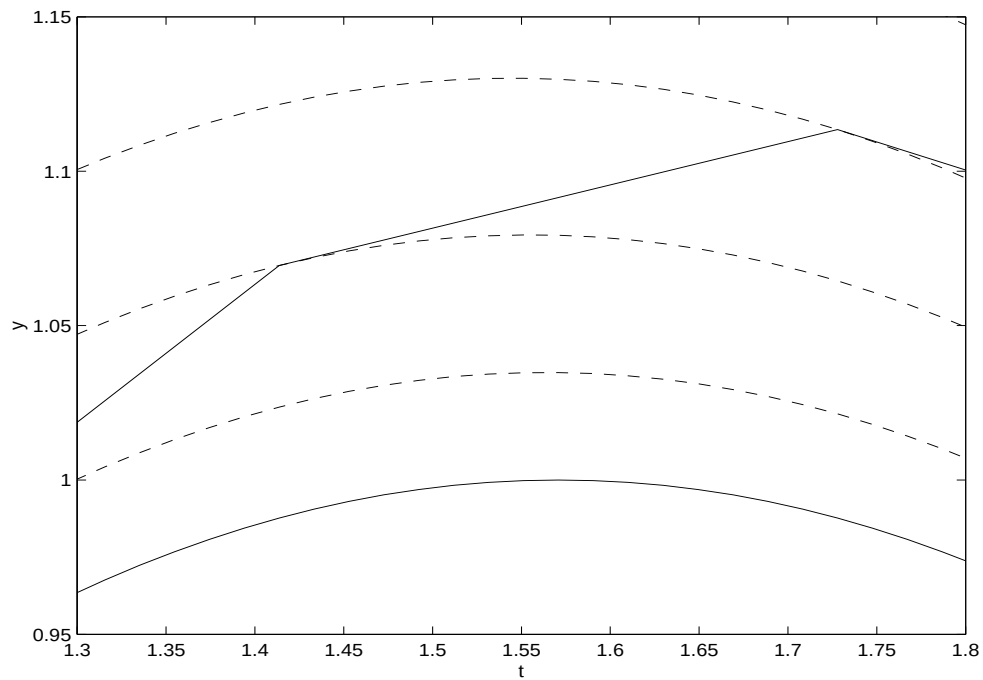
Implicit Midpoint Rule

$$y_{n+1} = y_n + hf \left(t_n + \frac{h}{2}, \frac{y_n + y_{n+1}}{2} \right)$$

Explicit Midpoint Rule

$$y_{n+1} = y_{n-1} + 2hf(t_n, y_n)$$

Local and global errors



Definitions

Global error:

$$g_n = y_n - y(t_n)$$

Local error:

$$\hat{l}_{n+1} = y_{n+1} - \hat{y}(t_{n+1})$$

Local error, explicit Euler

Let the local solution passing through (t_n, y_n) be $\hat{y}(t)$.
Compare explicit Euler to Taylor series expansion:

$$\begin{aligned} y_{n+1} &= \hat{y}(t_n) + hf(t_n, \hat{y}(t_n)) \\ - \quad \hat{y}(t_{n+1}) &= \hat{y}(t_n) + h\dot{\hat{y}}(t_n) + \frac{h^2}{2} \ddot{\hat{y}}(t_n) + \dots \end{aligned}$$

$$\hat{l}_{n+1} = -\frac{h^2}{2} \ddot{\hat{y}}(t_n) + \dots$$

The local error for explicit Euler is $\approx -\frac{h^2}{2} \ddot{\hat{y}}(t_n)$

Similarly,

The local error for implicit Euler is $\approx +\frac{h^2}{2} \ddot{\hat{y}}(t_n)$

Global error:

How are local errors propagated and accumulated?

Error propagation, explicit Euler

Explicit Euler method:

$$y_{n+1} = y_n + hf(t_n, y_n)$$

Subtract Taylor series expansion of exact solution:

$$\begin{aligned} y(t_{n+1}) &= y(t_n) + h\dot{y}(t_n) + \frac{h^2}{2}\ddot{y}(t_n) + \dots \\ &= y(t_n) + hf(t_n, y(t_n)) + \frac{h^2}{2}\ddot{y}(t_n) + \dots \end{aligned}$$

Global error recursion:

$$g_{n+1} = g_n + hf(t_n, y(t_n) + g_n) - hf(t_n, y(t_n)) + l_{n+1}$$

Take norms and use Lipschitz condition:

$$\|g_{n+1}\| \leq \|g_n\| + hL[f] \cdot \|g_n\| + \|l_{n+1}\|$$

Convergence, explicit Euler

Lemma: Let $\varepsilon_{n+1} \leq (1 + h\mu)\varepsilon_n + h\alpha$, with $h > 0$, $\mu > 0$, $\alpha > 0$ and $\varepsilon_0 = 0$. Then

$$\varepsilon_n \leq \alpha \cdot \frac{e^{t_n\mu} - 1}{\mu}; \quad t_n = n \cdot h$$

Apply to the global error recursion

$$\|g_{n+1}\| \leq \|g_n\| + hL[f] \cdot \|g_n\| + \|l_{n+1}\|$$

to obtain the **global error bound**

$$\|g_n\| \lesssim \frac{h \max \|\ddot{y}\|}{2} \cdot \frac{e^{t_n L[f]} - 1}{L[f]}$$

The explicit Euler method is convergent:

For $t_n = n \cdot h$ fixed, $\|g_n\| = O(h) \rightarrow 0$ as $h \rightarrow 0$.

Convergence order

Definition: If the global error satisfies

$$\|g_n\| = O(h^p)$$

then the method's *order of convergence* is p .

Note:

$$\begin{array}{ll} \text{local error} & \|l_n\| = O(h^{p+1}) \\ \text{global error} & \|g_n\| = O(h^p) \end{array} \quad \Rightarrow$$

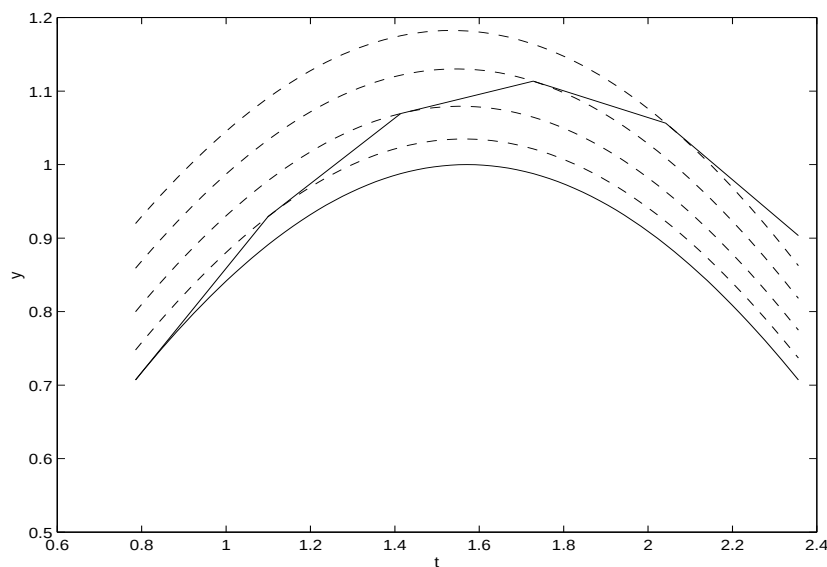
Examples:

- Explicit Euler $p = 1$
- Implicit Euler $p = 1$
- Trapezoidal Rule $p = 2$
- Implicit Midpoint Rule $p = 2$
- Explicit Midpoint Rule $p = 2$
- “Classical” Runge–Kutta method $p = 4$

Testing the Explicit Euler method

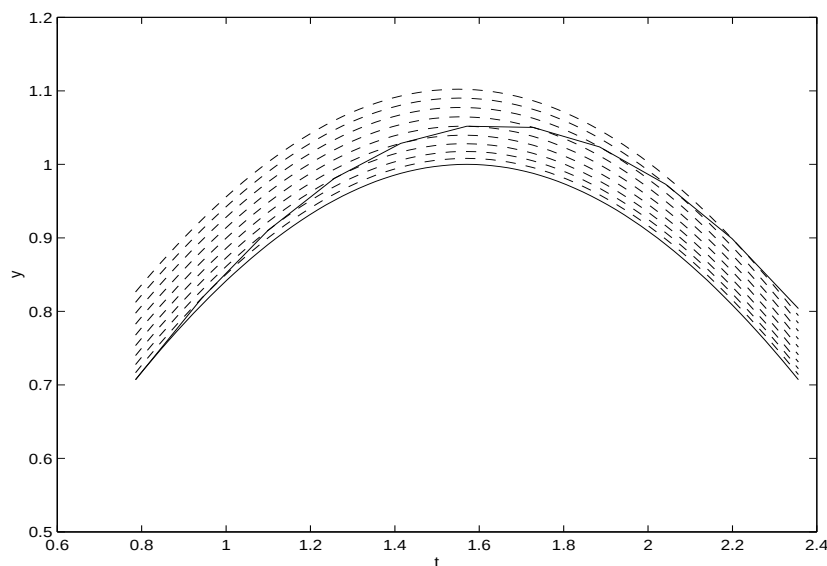
Solve $\dot{y} = \lambda(y - \sin t) + \cos t;$ $y(\pi/4) = 1/\sqrt{2}$

$$\lambda = -0.2$$



Explicit Euler

$$h = \pi/10$$



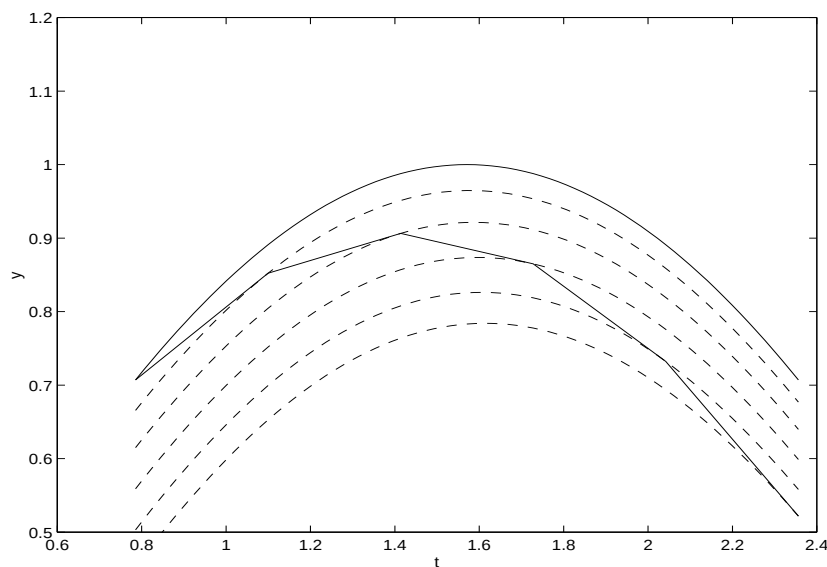
Explicit Euler

$$h = \pi/20$$

Testing the Implicit Euler method

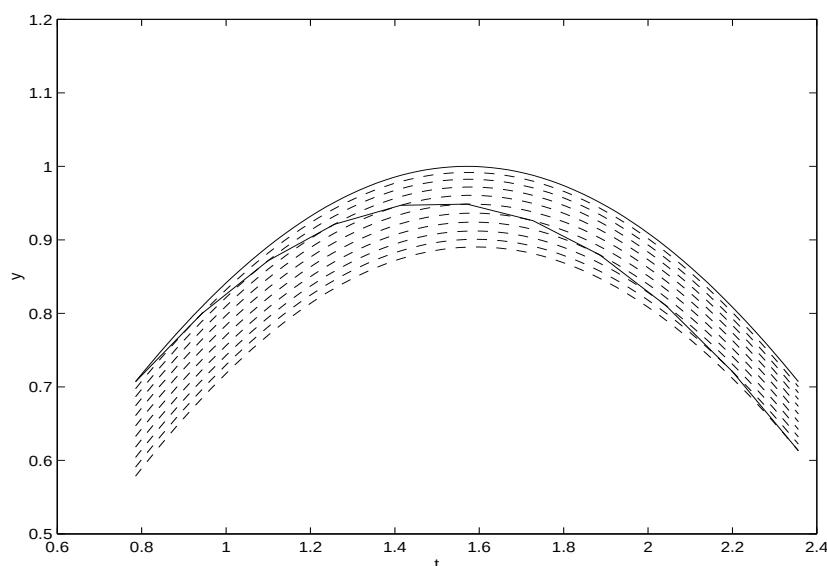
Solve $\dot{y} = \lambda(y - \sin t) + \cos t;$ $y(\pi/4) = 1/\sqrt{2}$

$$\lambda = -0.2$$



Implicit Euler

$$h = \pi/10$$



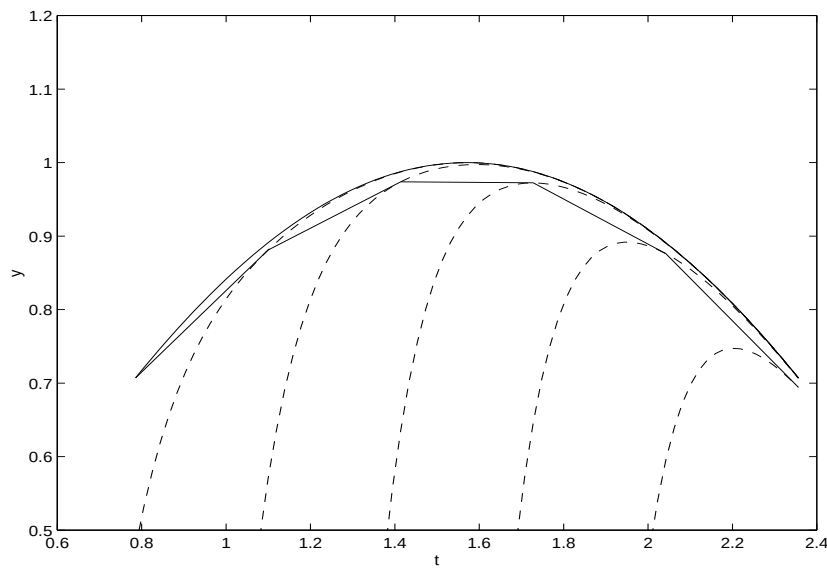
Implicit Euler

$$h = \pi/20$$

Testing the Implicit Euler method. . .

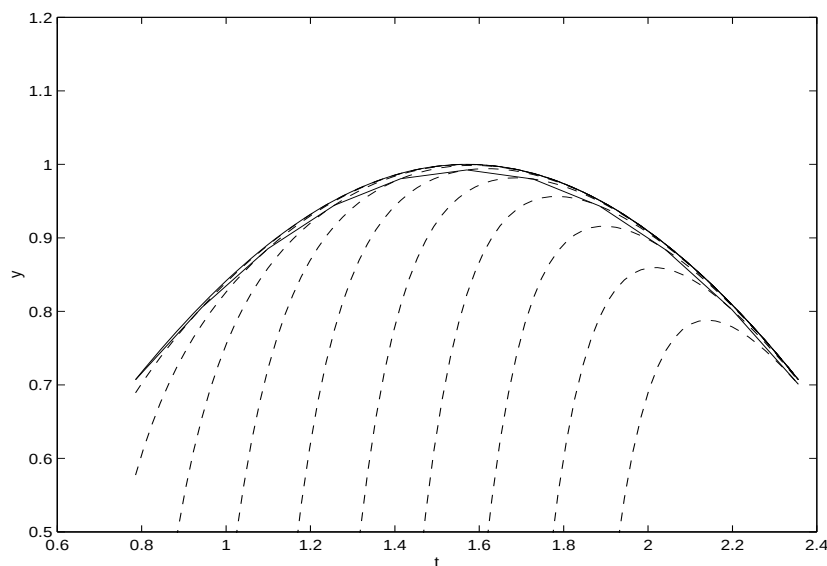
Solve $\dot{y} = \lambda(y - \sin t) + \cos t;$ $y(\pi/4) = 1/\sqrt{2}$

$$\lambda = -10$$



Implicit Euler

$$h = \pi/10$$



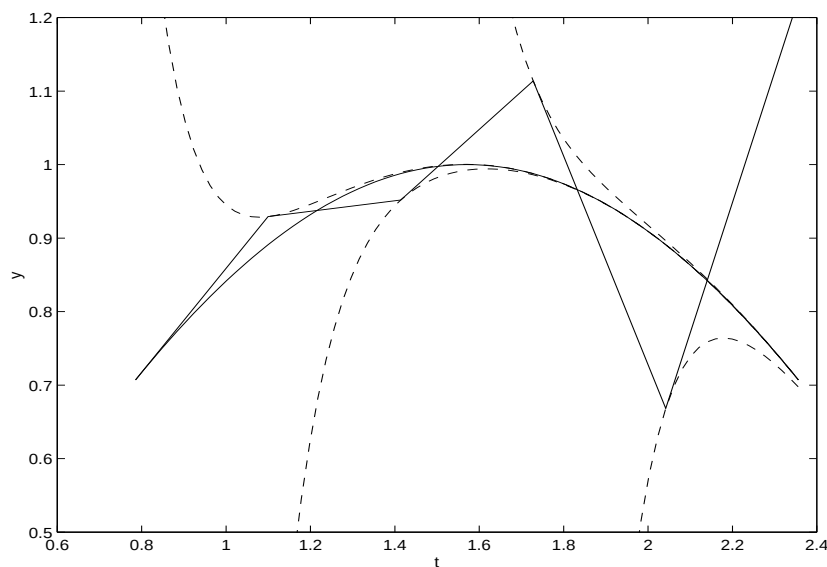
Implicit Euler

$$h = \pi/20$$

Testing the Explicit Euler method. . .

Solve $\dot{y} = \lambda(y - \sin t) + \cos t;$ $y(\pi/4) = 1/\sqrt{2}$

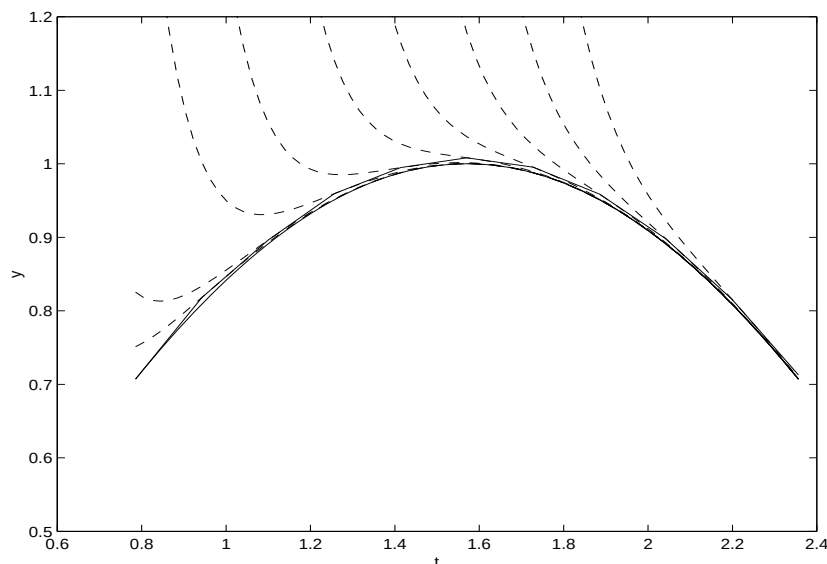
$$\lambda = -10$$



Explicit Euler

$$h = \pi/10$$

Instability!



Explicit Euler

$$h = \pi/20$$

Numerical instability

Example: Apply explicit Euler method to

$$\dot{y} = -100y; \quad y(t) = e^{-100t}y_0$$

Case 1 (Numerical instability): Stepsize $h = 0.1$ yields

$$y_{n+1} = (1 - 0.1 \cdot 100) \cdot y_n = -9y_n$$

$$y_n = (-9)^n y_0$$

Exponentially growing oscillations!

Case 2: Stepsize $h = 0.001$ yields

$$y_{n+1} = (1 - 0.001 \cdot 100) \cdot y_n = 0.9y_n$$

$$y_n = 0.9^n y_0$$

Smooth, decaying solution.

Stability analysis

“Test equation”: $\dot{y} = \lambda y$; solution $y(t) = e^{t\lambda}y_0$.

Mathematical stability condition:

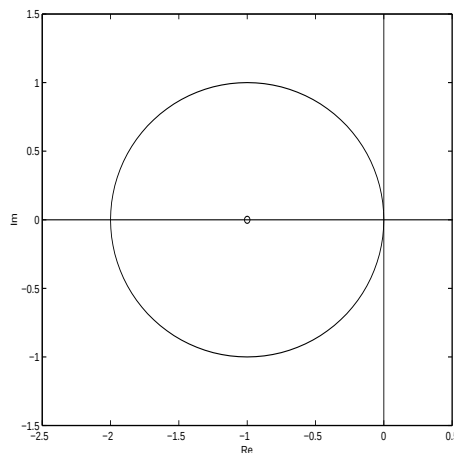
$$\text{Bounded solution} \Leftrightarrow \operatorname{Re} \lambda \leq 0$$

Explicit Euler: $y_{n+1} = (1 + h\lambda)y_n$

Numerical stability condition:

$$\text{Bounded solution} \Leftrightarrow |1 + h\lambda| \leq 1$$

Stability region: $\mathcal{S} = \{h\lambda \in \mathbb{C} : |1 + h\lambda| \leq 1\}$



Stability analysis. . .

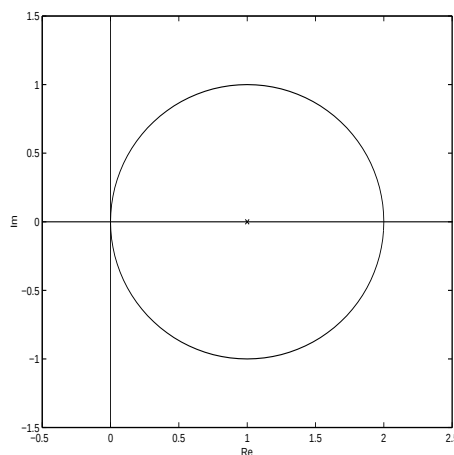
Apply implicit Euler method to linear test equation:

$$\begin{aligned}y_{n+1} &= y_n + h\lambda y_{n+1} \quad \Rightarrow \\y_{n+1} &= (1 - h\lambda)^{-1} y_n\end{aligned}$$

Numerical stability condition:

$$\text{Bounded solution} \Leftrightarrow |1 - h\lambda| \geq 1$$

Stability region: $\mathcal{S} = \{h\lambda \in \mathbb{C} : |1 - h\lambda| \geq 1\}$



Stiffness

Solve $\dot{y} = \lambda(y - \sin t) + \cos t$ **with** $\lambda = -50$.

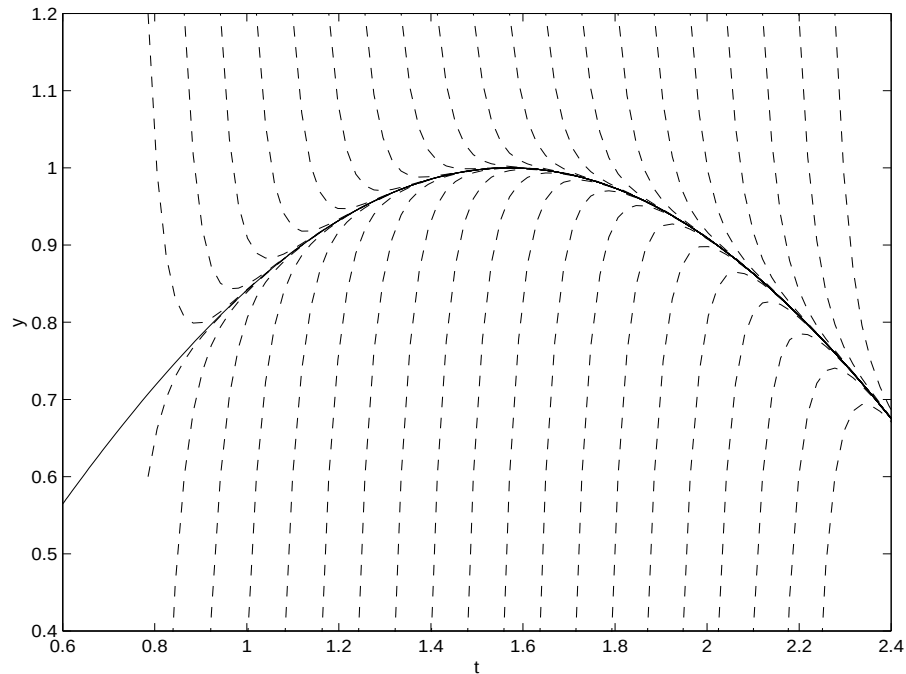
Solutions:

Particular: $y_P(t) = \sin t$

Homogeneous: $y_H(t) = e^{\lambda t}$

General: $y(t) = e^{\lambda(t-t_0)}(y(t_0) - \sin t_0) + \sin t$

Flow (solution trajectories):



Stiffness. . .

“Stiff” differential equations are characterized by homogeneous solutions being strongly damped.

Example: $\dot{y} = \lambda(y - \sin t) + \cos t$ with $\lambda \ll -1$.

Stability regions of explicit methods are bounded.
 $h\lambda \in \mathcal{S}$ puts strong restriction on h .

Example: Explicit Euler applied to the problem above requires $h \leq -2/\lambda \ll 1$.

Implicit methods with unbounded stability regions put no restrictions on h , *the stepsize is only restricted by accuracy requirement!*

Example: The implicit Euler applied to the problem above requires only that

$$l_n \approx \frac{h^2}{2} \sin t_n$$

is sufficiently small, independently of λ .

Runge–Kutta methods

Heun's method:

$$\dot{Y}_1 = f(t_n, y_n)$$

$$\dot{Y}_2 = f(t_n + h, y_n + h\dot{Y}_1)$$

$$y_{n+1} := y_n + \frac{h}{2} (\dot{Y}_1 + \dot{Y}_2)$$

Simple Runge–Kutta method of order $p = 2$.

Classical Runge–Kutta method

RK4 (1895) Order $p = 4$.

$$\dot{Y}_1 = f(t_n, y_n)$$

$$\dot{Y}_2 = f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}\dot{Y}_1\right)$$

$$\dot{Y}_3 = f\left(t_n + \frac{h}{2}, y_n + \frac{h}{2}\dot{Y}_2\right)$$

$$\dot{Y}_4 = f(t_n + h, y_n + h\dot{Y}_3)$$

$$y_{n+1} := y_n + \frac{h}{6} (\dot{Y}_1 + 2\dot{Y}_2 + 2\dot{Y}_3 + \dot{Y}_4)$$

Linear multistep methods

Example: Implicit Euler

$$y_{n+1} = y_n + hf(t_{n+1}, y_{n+1}) \quad \Rightarrow$$

$$y_{n+1} - y_n = hf(t_{n+1}, y_{n+1}) \quad \Rightarrow$$

$$\nabla y_{n+1} = hf(t_{n+1}, y_{n+1})$$

∇ is the **backward difference operator**.

Local error $l_n \approx h^2 \ddot{y}/2$, estimate by

$$l_n \approx \frac{h^2}{2} \cdot \frac{\nabla^2 y_{n+1}}{h^2}$$

New method: Compensate for the estimated error

$$\nabla y_{n+1} + \frac{\nabla^2}{2} y_{n+1} = hf(t_{n+1}, y_{n+1})$$

to get 2-step method of order $p = 2$.

Linear multistep methods. . .

Example: $\nabla y_{n+1} + \frac{\nabla^2}{2} y_{n+1} = hf(t_{n+1}, y_{n+1})$

$$\nabla y_{n+1} = y_{n+1} - y_n \quad \Rightarrow$$

$$\nabla^2 y_{n+1} = y_{n+1} - 2y_n + y_{n-1} \quad \Rightarrow$$

$$\frac{3}{2}y_{n+1} - 2y_n + \frac{1}{2}y_{n-1} = hf(t_{n+1}, y_{n+1})$$

General form:

$$\sum_{j=0}^k \alpha_{k-j} y_{n+1-j} = h \sum_{j=0}^k \beta_{k-j} f(t_{n+1-j}, y_{n+1-j})$$

Common methods:

- Adams-Moulton
- Backward differentiation methods (BDF)